

Instituto Federal do Sertão Pernambucano

Professor: Breno Leonardo

Disciplina: Programação II

Curso: Técnico em Informática

Aluno: _____

Matrícula: _____

Turma: 2º

Nota

Data: 20/05/2026

Marque o gabarito preenchendo completamente a região de cada alternativa.



	☐	☐	☐	☐	☐
	a	b	c	d	e
Q.1:	Discursiva				
Q.2:	☐	☐	☐	☐	☐
Q.3:	☐	☐	☐	☐	☐
Q.4:	☐	☐	☐	☐	☐
Q.5:	☐	☐	☐	☐	☐
Q.6:	☐	☐	☐	☐	☐
Q.7:	☐	☐	☐	☐	☐
Q.8:	☐	☐	☐	☐	☐
Q.9:	☐	☐	☐	☐	☐
Q.10:	☐	☐	☐	☐	☐
Q.11:	☐	☐	☐	☐	☐
Q.12:	☐	☐	☐	☐	☐
	a	b	c	d	e

Prova: 3035192.0

Q.1 (3.00) - Criar as classes Cliente, contaBancaria e ClassePrincipal. Um cliente pode ter de 0 a “n” contas. Uma conta não pode ser criada sem um cliente. Depois de criar as classes, implemente o exemplo na classe principal.

Q.2 (0.70) - Analise as afirmativas: I. O encapsulamento busca proteger os dados de uma classe.

II. Atributos privados podem ser acessados diretamente por qualquer classe.

III. Métodos getters e setters auxiliam no encapsulamento.

IV. O modificador private restringe o acesso aos atributos.

Assinale a alternativa correta:

- a) () Apenas I e II estão corretas
 b) () Apenas II e III estão corretas
 c) () Apenas I e IV estão corretas
 d) () Todas estão corretas
 e) () Apenas I, III e IV estão corretas

Verifique as respostas em: www.gradepen.com/?ansid=3035192.0

Q.3 (0.70) -

```
class Impressora {  
  
    void imprimir(String texto) {  
        System.out.println("Texto");  
    }  
  
    void imprimir(int numero) {  
        System.out.println("Número");  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Impressora i = new Impressora();  
  
        i.imprimir(10);  
    }  
}
```

Assinale a alternativa correta:

- a) ☐ O método imprimir não pode receber inteiro
- b) ☐ O código imprime Texto
- c) ☐ O código imprime Número
- d) ☐ Não existe sobrecarga no código
- e) ☐ O código imprime 10

Q.4 (0.60) - Em Java, métodos sobrecarregados devem possuir obrigatoriamente nomes diferentes.

- a) ☐ Certo
- b) ☐ Errado

Q.5 (0.70) -

```
abstract class Funcionario {  
  
    private String nome;  
  
    public String getNome() {  
        return nome;  
    }  
  
    public void setNome(String nome) {  
        this.nome = nome;  
    }  
  
    abstract double calcularSalario();  
}
```

Nesse código há exemplos de:

- a) ☐ Apenas herança
- b) ☐ Sobrecarga e sobrescrita
- c) ☐ Apenas encapsulamento
- d) ☐ Encapsulamento e abstração
- e) ☐ Apenas abstração

Q.6 (0.60) - A utilização de atributos privados (private) e métodos getters/setters é uma forma de implementar encapsulamento.

- a) ☐ Certo
- b) ☐ Errado

Q.7 (0.70) -

```
public class Main {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 3; i++) {  
            for (int j = 1; j <= 2; j++) {  
                System.out.println("Estudante " +  
                    i + ", Prova " + j);  
            }  
        }  
    }  
}
```

A execução do código a seguir será bem-sucedida e imprimirá, em tela, 6 linhas.

- a) ☐ Errado
- b) ☐ Certo

Q.8 (0.60) - Analise o código:

```
class Calculadora {  
  
    int soma(int a, int b) {  
        return a + b;  
    }  
  
    double soma(double a, double b) {  
        return a + b;  
    }  
}
```

Pode-se afirmar que o código apresenta:

- a) ☐ Classe abstrata
- b) ☐ Encapsulamento
- c) ☐ Herança múltipla
- d) ☐ Sobrescrita de métodos
- e) ☐ Sobrecarga de métodos

Q.9 (0.60) -

```
private String nome;

public void setNome(String nome) {
    this.nome = nome;
}

public String getNome() {
    return nome;
}
```

O atributo nome foi declarado como private para:

- a) () Restringir o acesso direto ao atributo
- b) () Impedir o encapsulamento
- c) () Permitir acesso direto por qualquer classe
- d) () Permitir herança múltipla
- e) () Tornar o atributo constante

Q.10 (0.60) - O principal objetivo do encapsulamento em Java é:

- a) () Permitir herança múltipla
- b) () Executar métodos mais rapidamente
- c) () Ocultar detalhes internos da implementação
- d) () Criar múltiplas classes simultaneamente
- e) () Eliminar construtores

Q.11 (0.60) - A abstração em orientação a objetos tem como objetivo:

- a) () Impedir reutilização de código
- b) () Ocultar características desnecessárias e mostrar apenas o essencial
- c) () Tornar atributos públicos
- d) () Expor todos os detalhes da implementação
- e) () Eliminar métodos da classe

Q.12 (0.60) -

```
class Impressora {

    void imprimir(String texto) { }

    void imprimir(int numero) { }

}
```

A diferença entre os métodos ocorre por:

- a) () Uso do construtor
- b) () Tipo de retorno B)
- c) () Modificador de acesso
- d) () Quantidade ou tipo dos parâmetros
- e) () Nome da classe

Instituto Federal do Sertão Pernambucano

Professor: Breno Leonardo

Disciplina: Programação II

Curso: Técnico em Informática

Aluno: _____

Matrícula: _____ Turma: 2º

Nota

Data: 20/05/2026

Marque o gabarito preenchendo completamente a região de cada alternativa.



	a	b	c	d	e
Q.1:					
Q.2:					
Q.3:					
Q.4:					
Q.5:					
Q.6:					
Q.7:					
Q.8:					
Q.9:					
Q.10:					
Q.11:					
Q.12:					

Prova: 3035192.1

Q.1 (3.00) - Criar as classes Cliente, contaBancaria e ClassePrincipal. Um cliente pode ter de 0 a “n” contas. Uma conta não pode ser criada sem um cliente. Depois de criar as classes, implemente o exemplo na classe principal.

Q.2 (0.60) -

```
private String nome;  
  
public void setName(String nome) {  
    this.nome = nome;  
}  
  
public String getName() {  
    return nome;  
}  
}
```

O atributo nome foi declarado como private para:

- a) () Impedir o encapsulamento
- b) () Permitir herança múltipla
- c) () Tornar o atributo constante
- d) () Restringir o acesso direto ao atributo

Verifique as respostas em: www.gradepen.com/?ansid=3035192.1

- e) ☐ Permitir acesso direto por qualquer classe

Q.3 (0.70) -

```
abstract class Funcionario {

    private String nome;

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    abstract double calcularSalario();
}
```

Nesse código há exemplos de:

- a) ☐ Apenas herança
 b) ☐ Sobrecarga e sobrescrita
 c) ☐ Apenas abstração
 d) ☐ Encapsulamento e abstração
 e) ☐ Apenas encapsulamento

Q.4 (0.60) - A utilização de atributos privados (private) e métodos getters/setters é uma forma de implementar encapsulamento.

- a) ☐ Errado
 b) ☐ Certo

Q.5 (0.70) -

```
class Impressora {

    void imprimir(String texto) {
        System.out.println("Texto");
    }

    void imprimir(int numero) {
        System.out.println("Número");
    }
}

public class Main {

    public static void main(String[] args) {

        Impressora i = new Impressora();

        i.imprimir(10);
    }
}
```

Assinale a alternativa correta:

- a) ☐ Não existe sobrecarga no código
 b) ☐ O código imprime Número
 c) ☐ O código imprime Texto
 d) ☐ O código imprime 10
 e) ☐ O método imprimir não pode receber inteiro

Q.6 (0.70) -

```
public class Main {
    public static void main(String[] args) {
        for (int i = 1; i <= 3; i++) {
            for (int j = 1; j <= 2; j++) {
                System.out.println("Estudante "
                    + i + ", Prova " + j);
            }
        }
    }
}
```

A execução do código a seguir será bem-sucedida e imprimirá, em tela, 6 linhas.

- a) ☐ Certo
 b) ☐ Errado

Q.7 (0.60) - Em Java, métodos sobrecarregados devem possuir obrigatoriamente nomes diferentes.

- a) ☐ Errado
 b) ☐ Certo

Q.8 (0.60) - A abstração em orientação a objetos tem como objetivo:

- a) () Ocultar características desnecessárias e mostrar apenas o essencial
- b) () Impedir reutilização de código
- c) () Eliminar métodos da classe
- d) () Expor todos os detalhes da implementação
- e) () Tornar atributos públicos

Q.9 (0.60) - O principal objetivo do encapsulamento em Java é:

- a) () Eliminar construtores
- b) () Criar múltiplas classes simultaneamente
- c) () Executar métodos mais rapidamente
- d) () Permitir herança múltipla
- e) () Ocultar detalhes internos da implementação

Q.10 (0.70) - Analise as afirmativas: I. O encapsulamento busca proteger os dados de uma classe.

II. Atributos privados podem ser acessados diretamente por qualquer classe.

III. Métodos getters e setters auxiliam no encapsulamento.

IV. O modificador private restringe o acesso aos atributos.

Assinale a alternativa correta:

- a) () Apenas I, III e IV estão corretas
- b) () Todas estão corretas
- c) () Apenas I e II estão corretas
- d) () Apenas I e IV estão corretas

- e) () Apenas II e III estão corretas

Q.11 (0.60) -

```
class Impressora {  
  
    void imprimir(String texto) { }  
  
    void imprimir(int numero) { }  
  
}
```

A diferença entre os métodos ocorre por:

- a) () Uso do construtor
- b) () Modificador de acesso
- c) () Tipo de retorno B)
- d) () Nome da classe
- e) () Quantidade ou tipo dos parâmetros

Q.12 (0.60) - Analise o código:

```
class Calculadora {  
  
    int soma(int a, int b) {  
        return a + b;  
    }  
  
    double soma(double a, double b) {  
        return a + b;  
    }  
  
}
```

Pode-se afirmar que o código apresenta:

- a) () Sobrecarga de métodos
- b) () Herança múltipla
- c) () Classe abstrata
- d) () Encapsulamento
- e) () Sobrescrita de métodos

Instituto Federal do Sertão Pernambucano

Professor: Breno Leonardo

Disciplina: Programação II

Curso: Técnico em Informática

Aluno: _____

Matrícula: _____

Turma: 2º

Nota

Data: 20/05/2026

Marque o gabarito preenchendo completamente a região de cada alternativa.



	a	b	c	d	e
Q.1:					
Q.2:					
Q.3:					
Q.4:					
Q.5:					
Q.6:					
Q.7:					
Q.8:					
Q.9:					
Q.10:					
Q.11:					
Q.12:					

Prova: 3035192.2

Q.1 (3.00) - Criar as classes Cliente, contaBancaria e ClassePrincipal. Um cliente pode ter de 0 a “n” contas. Uma conta não pode ser criada sem um cliente. Depois de criar as classes, implemente o exemplo na classe principal.

Q.2 (0.70) -

```
abstract class Funcionario {  
  
    private String nome;  
  
    public String getNome() {  
        return nome;  
    }  
  
    public void setNome(String nome) {  
        this.nome = nome;  
    }  
  
    abstract double calcularSalario();  
}
```

Nesse código há exemplos de:

a) () Apenas abstração

Verifique as respostas em: www.gradepen.com/?ansid=3035192.2

- b) () Encapsulamento e abstração
- c) () Apenas encapsulamento
- d) () Apenas herança
- e) () Sobrecarga e sobrescrita

Q.3 (0.60) - A abstração em orientação a objetos tem como objetivo:

- a) () Ocultar características desnecessárias e mostrar apenas o essencial
- b) () Expor todos os detalhes da implementação
- c) () Eliminar métodos da classe
- d) () Impedir reutilização de código
- e) () Tornar atributos públicos

Q.4 (0.60) - O principal objetivo do encapsulamento em Java é:

- a) () Criar múltiplas classes simultaneamente
- b) () Executar métodos mais rapidamente
- c) () Permitir herança múltipla
- d) () Eliminar construtores
- e) () Ocultar detalhes internos da implementação

Q.5 (0.60) - Analise o código:

```
class Calculadora {  
  
    int soma(int a, int b) {  
        return a + b;  
    }  
  
    double soma(double a, double b) {  
        return a + b;  
    }  
}
```

Pode-se afirmar que o código apresenta:

- a) () Classe abstrata
- b) () Encapsulamento
- c) () Herança múltipla
- d) () Sobrecarga de métodos
- e) () Sobrescrita de métodos

Q.6 (0.60) - Em Java, métodos sobrecarregados devem possuir obrigatoriamente nomes diferentes.

- a) () Errado
- b) () Certo

Q.7 (0.70) -

```
public class Main {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 3; i++) {  
            for (int j = 1; j <= 2; j++) {  
                System.out.println("Estudante " + i + ", Prova " + j);  
            }  
        }  
    }  
}
```

A execução do código a seguir será bem-sucedida e imprimirá, em tela, 6 linhas.

- a) () Errado
- b) () Certo

Q.8 (0.60) -

```
class Impressora {  
  
    void imprimir(String texto) { }  
  
    void imprimir(int numero) { }  
}
```

A diferença entre os métodos ocorre por:

- a) () Nome da classe
- b) () Tipo de retorno B)
- c) () Uso do construtor
- d) () Modificador de acesso
- e) () Quantidade ou tipo dos parâmetros

Q.9 (0.70) - Analise as afirmativas: I. O encapsulamento busca proteger os dados de uma classe.

II. Atributos privados podem ser acessados diretamente por qualquer classe.

III. Métodos getters e setters auxiliam no encapsulamento.

IV. O modificador private restringe o acesso aos atributos.

Assinale a alternativa correta:

- a) () Apenas II e III estão corretas
- b) () Apenas I, III e IV estão corretas
- c) () Todas estão corretas

- d) ☐ Apenas I e II estão corretas
e) ☐ Apenas I e IV estão corretas

Q.10 (0.60) - A utilização de atributos privados (private) e métodos getters/setters é uma forma de implementar encapsulamento.

- a) ☐ Errado
b) ☐ Certo

Q.11 (0.60) -

```
private String nome;  
  
public void setNome(String nome) {  
    this.nome = nome;  
}  
  
public String getNome() {  
    return nome;  
}  
}
```

O atributo nome foi declarado como private para:

- a) ☐ Permitir acesso direto por qualquer classe
b) ☐ Tornar o atributo constante
c) ☐ Impedir o encapsulamento
d) ☐ Permitir herança múltipla
e) ☐ Restringir o acesso direto ao atributo

Q.12 (0.70) -

```
class Impressora {  
  
    void imprimir(String texto) {  
        System.out.println("Texto");  
    }  
  
    void imprimir(int numero) {  
        System.out.println("Número");  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Impressora i = new Impressora();  
  
        i.imprimir(10);  
    }  
}
```

Assinale a alternativa correta:

- a) ☐ O código imprime Número
b) ☐ O código imprime 10
c) ☐ O código imprime Texto
d) ☐ Não existe sobrecarga no código
e) ☐ O método imprimir não pode receber inteiro

Instituto Federal do Sertão Pernambucano

Professor: Breno Leonardo

Disciplina: Programação II

Curso: Técnico em Informática

Aluno: _____

Matrícula: _____

Turma: 2º

Nota

Data: 20/05/2026

Marque o gabarito preenchendo completamente a região de cada alternativa.



	☐	☐	☐	☐	☐
	a	b	c	d	e
Q.1:	Discursiva				
Q.2:	☐	☐	☐	☐	☐
Q.3:	☐	☐	☐	☐	☐
Q.4:	☐	☐	☐	☐	☐
Q.5:	☐	☐	☐	☐	☐
Q.6:	☐	☐	☐	☐	☐
Q.7:	☐	☐	☐	☐	☐
Q.8:	☐	☐	☐	☐	☐
Q.9:	☐	☐	☐	☐	☐
Q.10:	☐	☐	☐	☐	☐
Q.11:	☐	☐	☐	☐	☐
Q.12:	☐	☐	☐	☐	☐
	a	b	c	d	e

Prova: 3035192.3

Q.1 (3.00) - Criar as classes Cliente, contaBancaria e ClassePrincipal. Um cliente pode ter de 0 a “n” contas. Uma conta não pode ser criada sem um cliente. Depois de criar as classes, implemente o exemplo na classe principal.

Q.2 (0.60) - Analise o código:

```
class Calculadora {

    int soma(int a, int b) {
        return a + b;
    }

    double soma(double a, double b) {
        return a + b;
    }
}
```

Pode-se afirmar que o código apresenta:

- a) () Sobrescrita de métodos
- b) () Sobrecarga de métodos
- c) () Classe abstrata
- d) () Encapsulamento

Verifique as respostas em: www.gradepen.com/?ansid=3035192.3

e) () Herança múltipla

Q.3 (0.60) -

```
class Impressora {  
  
    void imprimir(String texto) { }  
  
    void imprimir(int numero) { }  
  
}
```

A diferença entre os métodos ocorre por:

- a) () Quantidade ou tipo dos parâmetros
- b) () Modificador de acesso
- c) () Tipo de retorno B)
- d) () Uso do construtor
- e) () Nome da classe

Q.4 (0.70) -

```
public class Main {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 3; i++) {  
            for (int j = 1; j <= 2; j++) {  
                System.out.println("Estudante " +  
i + ", Prova " + j);  
            }  
        }  
    }  
}
```

A execução do código a seguir será bem-sucedida e imprimirá, em tela, 6 linhas.

- a) () Errado
- b) () Certo

Q.5 (0.70) -

```
class Impressora {  
  
    void imprimir(String texto) {  
        System.out.println("Texto");  
    }  
  
    void imprimir(int numero) {  
        System.out.println("Número");  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Impressora i = new Impressora();  
  
        i.imprimir(10);  
    }  
}
```

Assinale a alternativa correta:

- a) () O código imprime Texto
- b) () O método imprimir não pode receber inteiro
- c) () Não existe sobrecarga no código
- d) () O código imprime 10
- e) () O código imprime Número

Q.6 (0.60) - A abstração em orientação a objetos tem como objetivo:

- a) () Ocultar características desnecessárias e mostrar apenas o essencial
- b) () Tornar atributos públicos
- c) () Eliminar métodos da classe
- d) () Impedir reutilização de código
- e) () Expor todos os detalhes da implementação

Q.7 (0.60) - A utilização de atributos privados (private) e métodos getters/setters é uma forma de implementar encapsulamento.

- a) () Errado
- b) () Certo

Q.8 (0.60) - Em Java, métodos sobrecarregados devem possuir obrigatoriamente nomes diferentes.

- a) () Errado
- b) () Certo

Q.9 (0.70) -

```
abstract class Funcionario {  
  
    private String nome;  
  
    public String getNome() {  
        return nome;  
    }  
  
    public void setNome(String nome) {  
        this.nome = nome;  
    }  
  
    abstract double calcularSalario();  
}
```

Nesse código há exemplos de:

- a) ☐ Apenas abstração
- b) ☐ Apenas herança
- c) ☐ Apenas encapsulamento
- d) ☐ Encapsulamento e abstração
- e) ☐ Sobrecarga e sobrescrita

Q.10 (0.60) -

```
private String nome;  
  
public void setName(String nome) {  
    this.nome = nome;  
}  
  
public String getName() {  
    return nome;  
}  
}
```

O atributo nome foi declarado como private para:

- a) ☐ Permitir acesso direto por qualquer classe
- b) ☐ Tornar o atributo constante
- c) ☐ Restringir o acesso direto ao atributo
- d) ☐ Permitir herança múltipla
- e) ☐ Impedir o encapsulamento

Q.11 (0.60) - O principal objetivo do encapsulamento em Java é:

- a) ☐ Ocultar detalhes internos da implementação
- b) ☐ Eliminar construtores
- c) ☐ Criar múltiplas classes simultaneamente
- d) ☐ Permitir herança múltipla
- e) ☐ Executar métodos mais rapidamente

Q.12 (0.70) - Analise as afirmativas: I. O encapsulamento busca proteger os dados de uma classe.

II. Atributos privados podem ser acessados diretamente por qualquer classe.

III. Métodos getters e setters auxiliam no encapsulamento.

IV. O modificador private restringe o acesso aos atributos.

Assinale a alternativa correta:

- a) ☐ Todas estão corretas
- b) ☐ Apenas II e III estão corretas
- c) ☐ Apenas I, III e IV estão corretas
- d) ☐ Apenas I e II estão corretas
- e) ☐ Apenas I e IV estão corretas

Instituto Federal do Sertão Pernambucano

Professor: Breno Leonardo

Disciplina: Programação II

Curso: Técnico em Informática

Aluno: _____

Matrícula: _____

Turma: 2º

Nota

Data: 20/05/2026

Marque o gabarito preenchendo completamente a região de cada alternativa.



	☐	a	b	c	d	e	☐
Q.1:							
Q.2:							
Q.3:							
Q.4:							
Q.5:							
Q.6:							
Q.7:							
Q.8:							
Q.9:							
Q.10:							
Q.11:							
Q.12:							
		a	b	c	d	e	

Prova: 3035192.4

Q.1 (3.00) - Criar as classes Cliente, contaBancaria e ClassePrincipal. Um cliente pode ter de 0 a “n” contas. Uma conta não pode ser criada sem um cliente. Depois de criar as classes, implemente o exemplo na classe principal.

Q.2 (0.60) - A utilização de atributos privados (private) e métodos getters/setters é uma forma de implementar encapsulamento.

- a) () Certo
b) () Errado

Q.3 (0.70) -

Verifique as respostas em: www.gradepen.com/?ansid=3035192.4

```

class Impressora {

    void imprimir(String texto) {
        System.out.println("Texto");
    }

    void imprimir(int numero) {
        System.out.println("Número");
    }
}

public class Main {

    public static void main(String[] args) {

        Impressora i = new Impressora();

        i.imprimir(10);
    }
}

```

Assinale a alternativa correta:

- a) ☐ O código imprime Texto
- b) ☐ O código imprime 10
- c) ☐ O método imprimir não pode receber inteiro
- d) ☐ O código imprime Número
- e) ☐ Não existe sobrecarga no código

Q.4 (0.60) - Analise o código:

```

class Calculadora {

    int soma(int a, int b) {
        return a + b;
    }

    double soma(double a, double b) {
        return a + b;
    }
}

```

Pode-se afirmar que o código apresenta:

- a) ☐ Sobrescrita de métodos
- b) ☐ Encapsulamento
- c) ☐ Classe abstrata
- d) ☐ Herança múltipla
- e) ☐ Sobrecarga de métodos

Q.5 (0.60) -

```

private String nome;

public void setNome(String nome) {
    this.nome = nome;
}

public String getNome() {
    return nome;
}
}

```

O atributo nome foi declarado como private para:

- a) ☐ Permitir herança múltipla
- b) ☐ Tornar o atributo constante
- c) ☐ Permitir acesso direto por qualquer classe
- d) ☐ Impedir o encapsulamento
- e) ☐ Restringir o acesso direto ao atributo

Q.6 (0.60) - A abstração em orientação a objetos tem como objetivo:

- a) ☐ Eliminar métodos da classe
- b) ☐ Ocultar características desnecessárias e mostrar apenas o essencial
- c) ☐ Impedir reutilização de código
- d) ☐ Tornar atributos públicos
- e) ☐ Expor todos os detalhes da implementação

Q.7 (0.70) - Analise as afirmativas: I. O encapsulamento busca proteger os dados de uma classe.

II. Atributos privados podem ser acessados diretamente por qualquer classe.

III. Métodos getters e setters auxiliam no encapsulamento.

IV. O modificador private restringe o acesso aos atributos.

Assinale a alternativa correta:

- a) ☐ Apenas I e II estão corretas
- b) ☐ Apenas I e IV estão corretas
- c) ☐ Apenas I, III e IV estão corretas
- d) ☐ Apenas II e III estão corretas
- e) ☐ Todas estão corretas

Q.8 (0.60) - Em Java, métodos sobrecarregados devem possuir obrigatoriamente nomes diferentes.

- a) ☐ Certo
- b) ☐ Errado

Q.9 (0.70) -

```
abstract class Funcionario {  
  
    private String nome;  
  
    public String getNome() {  
        return nome;  
    }  
  
    public void setNome(String nome) {  
        this.nome = nome;  
    }  
  
    abstract double calcularSalario();  
}
```

Nesse código há exemplos de:

- a) ☐ Sobrecarga e sobrescrita
- b) ☐ Apenas encapsulamento
- c) ☐ Apenas abstração
- d) ☐ Apenas herança
- e) ☐ Encapsulamento e abstração

Q.10 (0.60) - O principal objetivo do encapsulamento em Java é:

- a) ☐ Permitir herança múltipla
- b) ☐ Executar métodos mais rapidamente
- c) ☐ Eliminar construtores

- d) ☐ Criar múltiplas classes simultaneamente
- e) ☐ Ocultar detalhes internos da implementação

Q.11 (0.60) -

```
class Impressora {  
  
    void imprimir(String texto) { }  
  
    void imprimir(int numero) { }  
  
}
```

A diferença entre os métodos ocorre por:

- a) ☐ Nome da classe
- b) ☐ Quantidade ou tipo dos parâmetros
- c) ☐ Modificador de acesso
- d) ☐ Tipo de retorno B)
- e) ☐ Uso do construtor

Q.12 (0.70) -

```
public class Main {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 3; i++) {  
            for (int j = 1; j <= 2; j++) {  
                System.out.println("Estudante "  
+ i + ", Prova " + j);  
            }  
        }  
    }  
}
```

A execução do código a seguir será bem-sucedida e imprimirá, em tela, 6 linhas.

- a) ☐ Errado
- b) ☐ Certo

Instituto Federal do Sertão Pernambucano

Professor: Breno Leonardo

Disciplina: Programação II

Curso: Técnico em Informática

Aluno: _____

Matrícula: _____

Turma: 2º

Nota

Data: 20/05/2026

Marque o gabarito preenchendo completamente a região de cada alternativa.	
	■ ■ a b c d e Q.1: Discursiva Q.2: ☐ ☐ ☐ ☐ ☐ Q.3: ☐ ☐ ☐ ☐ ☐ Q.4: ☐ ☐ ☐ ☐ ☐ Q.5: ☐ ☐ ☐ ☐ ☐ Q.6: ☐ ☐ ☐ ☐ ☐ Q.7: ☐ ☐ ☐ ☐ ☐ Q.8: ☐ ☐ ☐ ☐ ☐ Q.9: ☐ ☐ ☐ ☐ ☐ Q.10: ☐ ☐ ☐ ☐ ☐ Q.11: ☐ ☐ ☐ ☐ ☐ Q.12: ☐ ☐ ☐ ☐ ☐ a b c d e ■ ■
	Prova: 3035192.5

Q.1 (3.00) - Criar as classes Cliente, contaBancaria e ClassePrincipal. Um cliente pode ter de 0 a “n” contas. Uma conta não pode ser criada sem um cliente. Depois de criar as classes, implemente o exemplo na classe principal.

Q.2 (0.60) - A abstração em orientação a objetos tem como objetivo:

- a) () Impedir reutilização de código
- b) () Eliminar métodos da classe
- c) () Ocultar características desnecessárias e mostrar apenas o essencial
- d) () Tornar atributos públicos
- e) () Expor todos os detalhes da implementação

Q.3 (0.70) -

```

abstract class Funcionario {

    private String nome;

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    abstract double calcularSalario();
}

```

Nesse código há exemplos de:

- a) ☐ Encapsulamento e abstração
- b) ☐ Sobrecarga e sobrescrita
- c) ☐ Apenas encapsulamento
- d) ☐ Apenas herança
- e) ☐ Apenas abstração

Q.4 (0.60) - O principal objetivo do encapsulamento em Java é:

- a) ☐ Executar métodos mais rapidamente
- b) ☐ Ocultar detalhes internos da implementação
- c) ☐ Permitir herança múltipla
- d) ☐ Criar múltiplas classes simultaneamente
- e) ☐ Eliminar construtores

Q.5 (0.60) - A utilização de atributos privados (private) e métodos getters/setters é uma forma de implementar encapsulamento.

- a) ☐ Errado
- b) ☐ Certo

Q.6 (0.60) -

```

private String nome;

public void setNome(String nome) {
    this.nome = nome;
}

public String getNome() {
    return nome;
}

```

O atributo nome foi declarado como private para:

- a) ☐ Impedir o encapsulamento
- b) ☐ Permitir herança múltipla
- c) ☐ Permitir acesso direto por qualquer classe
- d) ☐ Tornar o atributo constante
- e) ☐ Restringir o acesso direto ao atributo

Q.7 (0.60) - Analise o código:

```

class Calculadora {

    int soma(int a, int b) {
        return a + b;
    }

    double soma(double a, double b) {
        return a + b;
    }
}

```

Pode-se afirmar que o código apresenta:

- a) ☐ Herança múltipla
- b) ☐ Sobrecarga de métodos
- c) ☐ Classe abstrata
- d) ☐ Sobrescrita de métodos
- e) ☐ Encapsulamento

Q.8 (0.70) -

```

public class Main {
    public static void main(String[] args) {
        for (int i = 1; i <= 3; i++) {
            for (int j = 1; j <= 2; j++) {
                System.out.println("Estudante "
                    + i + ", Prova " + j);
            }
        }
    }
}

```

A execução do código a seguir será bem-sucedida e imprimirá, em tela, 6 linhas.

- a) ☐ Certo
- b) ☐ Errado

Q.9 (0.70) - Analise as afirmativas: I. O encapsulamento busca proteger os dados de uma classe.

II. Atributos privados podem ser acessados diretamente por qualquer classe.

III. Métodos getters e setters auxiliam no encapsulamento.

IV. O modificador `private` restringe o acesso aos atributos.

Assinale a alternativa correta:

- a) ☐ Apenas I e II estão corretas
- b) ☐ Apenas I e IV estão corretas
- c) ☐ Apenas I, III e IV estão corretas
- d) ☐ Todas estão corretas
- e) ☐ Apenas II e III estão corretas

Q.10 (0.60) -

```
class Impressora {  
  
    void imprimir(String texto) { }  
  
    void imprimir(int numero) { }  
  
}
```

A diferença entre os métodos ocorre por:

- a) ☐ Uso do construtor
- b) ☐ Tipo de retorno B)
- c) ☐ Modificador de acesso
- d) ☐ Nome da classe
- e) ☐ Quantidade ou tipo dos parâmetros

Q.11 (0.70) -

```
class Impressora {  
  
    void imprimir(String texto) {  
        System.out.println("Texto");  
    }  
  
    void imprimir(int numero) {  
        System.out.println("Número");  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Impressora i = new Impressora();  
  
        i.imprimir(10);  
    }  
}
```

Assinale a alternativa correta:

- a) ☐ O método `imprimir` não pode receber inteiro
- b) ☐ O código imprime Número
- c) ☐ O código imprime Texto
- d) ☐ O código imprime 10
- e) ☐ Não existe sobrecarga no código

Q.12 (0.60) - Em Java, métodos sobrecarregados devem possuir obrigatoriamente nomes diferentes.

- a) ☐ Certo
- b) ☐ Errado

Instituto Federal do Sertão Pernambucano

Professor: Breno Leonardo

Disciplina: Programação II

Curso: Técnico em Informática

Aluno: _____

Matrícula: _____

Turma: 2º

Nota

Data: 20/05/2026

Marque o gabarito preenchendo completamente a região de cada alternativa.



	☐	a	☐	b	☐	c	☐	d	☐	e	☐
Q.1:		Discursiva									
Q.2:		☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
Q.3:		☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
Q.4:		☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
Q.5:		☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
Q.6:		☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
Q.7:		☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
Q.8:		☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
Q.9:		☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
Q.10:		☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
Q.11:		☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
Q.12:		☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
		a	b	c	d	e					

Prova: 3035192.6

Q.1 (3.00) - Criar as classes Cliente, contaBancaria e ClassePrincipal. Um cliente pode ter de 0 a “n” contas. Uma conta não pode ser criada sem um cliente. Depois de criar as classes, implemente o exemplo na classe principal.

Q.2 (0.60) -

```
private String nome;  
  
public void setName(String nome) {  
    this.nome = nome;  
}  
  
public String getName() {  
    return nome;  
}  
}
```

O atributo nome foi declarado como private para:

- a) () Restringir o acesso direto ao atributo
- b) () Permitir acesso direto por qualquer classe
- c) () Tornar o atributo constante

- d) ☐ Permitir herança múltipla
e) ☐ Impedir o encapsulamento

Q.3 (0.70) -

```
public class Main {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 3; i++) {  
            for (int j = 1; j <= 2; j++) {  
                System.out.println("Estudante " +  
i + " ", Prova " + j);  
            }  
        }  
    }  
}
```

A execução do código a seguir será bem-sucedida e imprimirá, em tela, 6 linhas.

- a) ☐ Errado
b) ☐ Certo

Q.4 (0.70) -

```
abstract class Funcionario {  
  
    private String nome;  
  
    public String getNome() {  
        return nome;  
    }  
  
    public void setNome(String nome) {  
        this.nome = nome;  
    }  
  
    abstract double calcularSalario();  
}
```

Nesse código há exemplos de:

- a) ☐ Apenas abstração
b) ☐ Apenas encapsulamento
c) ☐ Apenas herança
d) ☐ Sobrecarga e sobrescrita
e) ☐ Encapsulamento e abstração

Q.5 (0.60) - O principal objetivo do encapsulamento em Java é:

- a) ☐ Permitir herança múltipla
b) ☐ Ocultar detalhes internos da implementação
c) ☐ Eliminar construtores
d) ☐ Criar múltiplas classes simultaneamente
e) ☐ Executar métodos mais rapidamente

Q.6 (0.60) - A utilização de atributos privados (private) e métodos getters/setters é uma forma de implementar encapsulamento.

- a) ☐ Errado
b) ☐ Certo

Q.7 (0.60) - Em Java, métodos sobrecarregados devem possuir obrigatoriamente nomes diferentes.

- a) ☐ Certo
b) ☐ Errado

Q.8 (0.70) -

```
class Impressora {  
  
    void imprimir(String texto) {  
        System.out.println("Texto");  
    }  
  
    void imprimir(int numero) {  
        System.out.println("Número");  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Impressora i = new Impressora();  
  
        i.imprimir(10);  
    }  
}
```

Assinale a alternativa correta:

- a) ☐ O código imprime 10
b) ☐ O código imprime Número
c) ☐ Não existe sobrecarga no código
d) ☐ O código imprime Texto
e) ☐ O método imprimir não pode receber inteiro

Q.9 (0.60) - A abstração em orientação a objetos tem como objetivo:

- a) ☐ Ocultar características desnecessárias e mostrar apenas o essencial
b) ☐ Tornar atributos públicos
c) ☐ Expor todos os detalhes da implementação

- d) () Eliminar métodos da classe
e) () Impedir reutilização de código

Q.10 (0.60) - Analise o código:

```
class Calculadora {  
  
    int soma(int a, int b) {  
        return a + b;  
    }  
  
    double soma(double a, double b) {  
        return a + b;  
    }  
}
```

Pode-se afirmar que o código apresenta:

- a) () Sobrecarga de métodos
b) () Sobrescrita de métodos
c) () Classe abstrata
d) () Herança múltipla
e) () Encapsulamento

Q.11 (0.70) - Analise as afirmativas: I. O encapsulamento busca proteger os dados de uma classe.

II. Atributos privados podem ser acessados diretamente por qualquer classe.

III. Métodos getters e setters auxiliam no encapsulamento.

IV. O modificador private restringe o acesso aos atributos.

Assinale a alternativa correta:

- a) () Apenas II e III estão corretas
b) () Apenas I e II estão corretas
c) () Apenas I e IV estão corretas
d) () Apenas I, III e IV estão corretas
e) () Todas estão corretas

Q.12 (0.60) -

```
class Impressora {  
  
    void imprimir(String texto) { }  
  
    void imprimir(int numero) { }  
}
```

A diferença entre os métodos ocorre por:

- a) () Uso do construtor
b) () Modificador de acesso
c) () Tipo de retorno B)
d) () Nome da classe
e) () Quantidade ou tipo dos parâmetros

Instituto Federal do Sertão Pernambucano

Professor: Breno Leonardo

Disciplina: Programação II

Curso: Técnico em Informática

Aluno: _____

Matrícula: _____

Turma: 2º

Nota

Data: 20/05/2026

Marque o gabarito preenchendo completamente a região de cada alternativa.



	a	b	c	d	e
Q.1:					
Q.2:					
Q.3:					
Q.4:					
Q.5:					
Q.6:					
Q.7:					
Q.8:					
Q.9:					
Q.10:					
Q.11:					
Q.12:					

Prova: 3035192.7

Q.1 (3.00) - Criar as classes Cliente, contaBancaria e ClassePrincipal. Um cliente pode ter de 0 a “n” contas. Uma conta não pode ser criada sem um cliente. Depois de criar as classes, implemente o exemplo na classe principal.

Q.2 (0.60) - O principal objetivo do encapsulamento em Java é:

- a) () Executar métodos mais rapidamente
- b) () Eliminar construtores
- c) () Ocultar detalhes internos da implementação
- d) () Criar múltiplas classes simultaneamente
- e) () Permitir herança múltipla

Q.3 (0.60) -

```
class Impressora {  
  
    void imprimir(String texto) { }  
  
    void imprimir(int numero) { }  
  
}
```

A diferença entre os métodos ocorre por:

- a) ☐ Tipo de retorno B)
- b) ☐ Modificador de acesso
- c) ☐ Uso do construtor
- d) ☐ Quantidade ou tipo dos parâmetros
- e) ☐ Nome da classe

Q.4 (0.60) - Analise o código:

```
class Calculadora {

    int soma(int a, int b) {
        return a + b;
    }

    double soma(double a, double b) {
        return a + b;
    }
}
```

Pode-se afirmar que o código apresenta:

- a) ☐ Herança múltipla
- b) ☐ Sobrescrita de métodos
- c) ☐ Encapsulamento
- d) ☐ Sobrecarga de métodos
- e) ☐ Classe abstrata

Q.5 (0.60) -

```
private String nome;

public void setNome(String nome) {
    this.nome = nome;
}

public String getNome() {
    return nome;
}
}
```

O atributo nome foi declarado como private para:

- a) ☐ Permitir herança múltipla
- b) ☐ Tornar o atributo constante
- c) ☐ Impedir o encapsulamento
- d) ☐ Permitir acesso direto por qualquer classe
- e) ☐ Restringir o acesso direto ao atributo

Q.6 (0.60) - Em Java, métodos sobrecarregados devem possuir obrigatoriamente nomes diferentes.

- a) ☐ Certo
- b) ☐ Errado

Q.7 (0.70) -

```
public class Main {
    public static void main(String[] args) {
        for (int i = 1; i <= 3; i++) {
            for (int j = 1; j <= 2; j++) {
                System.out.println("Estudante "
                    + i + ", Prova " + j);
            }
        }
    }
}
```

A execução do código a seguir será bem-sucedida e imprimirá, em tela, 6 linhas.

- a) ☐ Certo
- b) ☐ Errado

Q.8 (0.60) - A abstração em orientação a objetos tem como objetivo:

- a) ☐ Eliminar métodos da classe
- b) ☐ Impedir reutilização de código
- c) ☐ Ocultar características desnecessárias e mostrar apenas o essencial
- d) ☐ Expor todos os detalhes da implementação
- e) ☐ Tornar atributos públicos

Q.9 (0.70) -

```
abstract class Funcionario {

    private String nome;

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    abstract double calcularSalario();
}
```

Nesse código há exemplos de:

- a) ☐ Apenas abstração
- b) ☐ Apenas herança

- c) ☐ Apenas encapsulamento
- d) ☐ Encapsulamento e abstração
- e) ☐ Sobrecarga e sobrescrita

Q.10 (0.60) - A utilização de atributos privados (private) e métodos getters/setters é uma forma de implementar encapsulamento.

- a) ☐ Errado
- b) ☐ Certo

Q.11 (0.70) -

```
class Impressora {  
  
    void imprimir(String texto) {  
        System.out.println("Texto");  
    }  
  
    void imprimir(int numero) {  
        System.out.println("Número");  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Impressora i = new Impressora();  
  
        i.imprimir(10);  
    }  
}
```

Assinale a alternativa correta:

- a) ☐ O código imprime Texto
- b) ☐ Não existe sobrecarga no código
- c) ☐ O código imprime 10
- d) ☐ O método imprimir não pode receber inteiro
- e) ☐ O código imprime Número

Q.12 (0.70) - Analise as afirmativas: I. O encapsulamento busca proteger os dados de uma classe.

II. Atributos privados podem ser acessados diretamente por qualquer classe.

III. Métodos getters e setters auxiliam no encapsulamento.

IV. O modificador private restringe o acesso aos atributos.

Assinale a alternativa correta:

- a) ☐ Apenas I e IV estão corretas
- b) ☐ Apenas I, III e IV estão corretas
- c) ☐ Apenas II e III estão corretas
- d) ☐ Todas estão corretas
- e) ☐ Apenas I e II estão corretas

Instituto Federal do Sertão Pernambucano

Professor: Breno Leonardo

Disciplina: Programação II

Curso: Técnico em Informática

Aluno: _____

Matrícula: _____

Turma: 2º

Nota

Data: 20/05/2026

Marque o gabarito preenchendo completamente a região de cada alternativa.



	☐	a	b	c	d	e	☐
Q.1:		Discursiva					
Q.2:	☐	☐	☐	☐	☐	☐	☐
Q.3:	☐	☐	☐	☐	☐	☐	☐
Q.4:	☐	☐	☐	☐	☐	☐	☐
Q.5:	☐	☐	☐	☐	☐	☐	☐
Q.6:	☐	☐	☐	☐	☐	☐	☐
Q.7:	☐	☐	☐	☐	☐	☐	☐
Q.8:	☐	☐	☐	☐	☐	☐	☐
Q.9:	☐	☐	☐	☐	☐	☐	☐
Q.10:	☐	☐	☐	☐	☐	☐	☐
Q.11:	☐	☐	☐	☐	☐	☐	☐
Q.12:	☐	☐	☐	☐	☐	☐	☐
		a	b	c	d	e	

Prova: 3035192.8

Q.1 (3.00) - Criar as classes Cliente, contaBancaria e ClassePrincipal. Um cliente pode ter de 0 a “n” contas. Uma conta não pode ser criada sem um cliente. Depois de criar as classes, implemente o exemplo na classe principal.

Q.2 (0.70) -

```
public class Main {
    public static void main(String[] args) {
        for (int i = 1; i <= 3; i++) {
            for (int j = 1; j <= 2; j++) {
                System.out.println("Estudante "
                    + i + ", Prova " + j);
            }
        }
    }
}
```

A execução do código a seguir será bem-sucedida e imprimirá, em tela, 6 linhas.

a) () Errado

b) () Certo

Q.3 (0.60) - A utilização de atributos privados (private) e métodos getters/setters é uma forma de implementar encapsulamento.

- a) ☐ Certo
b) ☐ Errado

Q.4 (0.60) - Em Java, métodos sobrecarregados devem possuir obrigatoriamente nomes diferentes.

- a) ☐ Errado
b) ☐ Certo

Q.5 (0.60) - O principal objetivo do encapsulamento em Java é:

- a) ☐ Permitir herança múltipla
b) ☐ Executar métodos mais rapidamente
c) ☐ Eliminar construtores
d) ☐ Criar múltiplas classes simultaneamente
e) ☐ Ocultar detalhes internos da implementação

Q.6 (0.60) -

```
class Impressora {  
  
    void imprimir(String texto) { }  
  
    void imprimir(int numero) { }  
  
}
```

A diferença entre os métodos ocorre por:

- a) ☐ Quantidade ou tipo dos parâmetros
b) ☐ Modificador de acesso
c) ☐ Nome da classe
d) ☐ Tipo de retorno B)
e) ☐ Uso do construtor

Q.7 (0.60) - A abstração em orientação a objetos tem como objetivo:

- a) ☐ Ocultar características desnecessárias e mostrar apenas o essencial
b) ☐ Eliminar métodos da classe
c) ☐ Impedir reutilização de código
d) ☐ Expor todos os detalhes da implementação
e) ☐ Tornar atributos públicos

Q.8 (0.70) - Analise as afirmativas: I. O encapsulamento busca proteger os dados de uma classe.

II. Atributos privados podem ser acessados diretamente por qualquer classe.

III. Métodos getters e setters auxiliam no encapsulamento.

IV. O modificador private restringe o acesso aos atributos.

Assinale a alternativa correta:

- a) ☐ Apenas I, III e IV estão corretas
b) ☐ Todas estão corretas
c) ☐ Apenas II e III estão corretas
d) ☐ Apenas I e IV estão corretas
e) ☐ Apenas I e II estão corretas

Q.9 (0.70) -

```
class Impressora {  
  
    void imprimir(String texto) {  
        System.out.println("Texto");  
    }  
  
    void imprimir(int numero) {  
        System.out.println("Número");  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Impressora i = new Impressora();  
  
        i.imprimir(10);  
    }  
}
```

Assinale a alternativa correta:

- a) ☐ O código imprime 10
b) ☐ O código imprime Texto
c) ☐ O código imprime Número
d) ☐ O método imprimir não pode receber inteiro
e) ☐ Não existe sobrecarga no código

Q.10 (0.60) - Analise o código:

```
class Calculadora {  
  
    int soma(int a, int b) {  
        return a + b;  
    }  
  
    double soma(double a, double b) {  
        return a + b;  
    }  
}
```

Pode-se afirmar que o código apresenta:

- a) () Herança múltipla
- b) () Sobrescrita de métodos
- c) () Sobrecarga de métodos
- d) () Classe abstrata
- e) () Encapsulamento

Q.11 (0.70) -

```
abstract class Funcionario {  
  
    private String nome;  
  
    public String getNome() {  
        return nome;  
    }  
  
    public void setNome(String nome) {  
        this.nome = nome;  
    }  
  
    abstract double calcularSalario();  
}
```

Nesse código há exemplos de:

- a) () Encapsulamento e abstração
- b) () Sobrecarga e sobrescrita
- c) () Apenas abstração
- d) () Apenas herança
- e) () Apenas encapsulamento

Q.12 (0.60) -

```
private String nome;  
  
public void setNome(String nome) {  
    this.nome = nome;  
}  
  
public String getNome() {  
    return nome;  
}  
}
```

O atributo nome foi declarado como private para:

- a) () Permitir acesso direto por qualquer classe
- b) () Tornar o atributo constante
- c) () Permitir herança múltipla
- d) () Restringir o acesso direto ao atributo
- e) () Impedir o encapsulamento

Instituto Federal do Sertão Pernambucano

Professor: Breno Leonardo

Disciplina: Programação II

Curso: Técnico em Informática

Aluno: _____

Matrícula: _____

Turma: 2º

Nota

Data: 20/05/2026

Marque o gabarito preenchendo completamente a região de cada alternativa.	
	a b c d e Q.1: Discursiva Q.2: ☐ ☐ ☐ ☐ ☐ Q.3: ☐ ☐ ☐ ☐ ☐ Q.4: ☐ ☐ ☐ ☐ ☐ Q.5: ☐ ☐ ☐ ☐ ☐ Q.6: ☐ ☐ ☐ ☐ ☐ Q.7: ☐ ☐ ☐ ☐ ☐ Q.8: ☐ ☐ ☐ ☐ ☐ Q.9: ☐ ☐ ☐ ☐ ☐ Q.10: ☐ ☐ ☐ ☐ ☐ Q.11: ☐ ☐ ☐ ☐ ☐ Q.12: ☐ ☐ ☐ ☐ ☐ a b c d e
	Prova: 3035192.9

Q.1 (3.00) - Criar as classes Cliente, contaBancaria e ClassePrincipal. Um cliente pode ter de 0 a “n” contas. Uma conta não pode ser criada sem um cliente. Depois de criar as classes, implemente o exemplo na classe principal.

Q.2 (0.60) - A abstração em orientação a objetos tem como objetivo:

- a) () Impedir reutilização de código
- b) () Eliminar métodos da classe
- c) () Expor todos os detalhes da implementação
- d) () Ocultar características desnecessárias e mostrar apenas o essencial
- e) () Tornar atributos públicos

Q.3 (0.60) - Analise o código:

```
class Calculadora {

    int soma(int a, int b) {
        return a + b;
    }

    double soma(double a, double b) {
        return a + b;
    }
}
```

Pode-se afirmar que o código apresenta:

- a) ☐ Classe abstrata
- b) ☐ Sobrecarga de métodos
- c) ☐ Encapsulamento
- d) ☐ Herança múltipla
- e) ☐ Sobrescrita de métodos

Q.4 (0.60) - O principal objetivo do encapsulamento em Java é:

- a) ☐ Permitir herança múltipla
- b) ☐ Criar múltiplas classes simultaneamente
- c) ☐ Ocultar detalhes internos da implementação
- d) ☐ Eliminar construtores
- e) ☐ Executar métodos mais rapidamente

Q.5 (0.60) -

```
class Impressora {

    void imprimir(String texto) { }

    void imprimir(int numero) { }

}
```

A diferença entre os métodos ocorre por:

- a) ☐ Tipo de retorno B)
- b) ☐ Uso do construtor
- c) ☐ Quantidade ou tipo dos parâmetros
- d) ☐ Modificador de acesso
- e) ☐ Nome da classe

Q.6 (0.60) -

```
private String nome;

public void setNome(String nome) {
    this.nome = nome;
}

public String getNome() {
    return nome;
}
}
```

O atributo nome foi declarado como private para:

- a) ☐ Impedir o encapsulamento
- b) ☐ Permitir acesso direto por qualquer classe
- c) ☐ Permitir herança múltipla
- d) ☐ Tornar o atributo constante
- e) ☐ Restringir o acesso direto ao atributo

Q.7 (0.60) - A utilização de atributos privados (private) e métodos getters/setters é uma forma de implementar encapsulamento.

- a) ☐ Errado
- b) ☐ Certo

Q.8 (0.70) -

```
abstract class Funcionario {

    private String nome;

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    abstract double calcularSalario();
}
```

Nesse código há exemplos de:

- a) ☐ Sobrecarga e sobrescrita
- b) ☐ Apenas herança
- c) ☐ Apenas encapsulamento
- d) ☐ Apenas abstração
- e) ☐ Encapsulamento e abstração

Q.9 (0.60) - Em Java, métodos sobrecarregados devem possuir obrigatoriamente nomes diferentes.

- a) ☐ Errado
- b) ☐ Certo

Q.10 (0.70) - Analise as afirmativas: I. O encapsulamento busca proteger os dados de uma classe.

II. Atributos privados podem ser acessados diretamente por qualquer classe.

III. Métodos getters e setters auxiliam no encapsulamento.

IV. O modificador private restringe o acesso aos atributos.

Assinale a alternativa correta:

- a) ☐ Apenas I, III e IV estão corretas
- b) ☐ Apenas I e II estão corretas
- c) ☐ Todas estão corretas
- d) ☐ Apenas II e III estão corretas
- e) ☐ Apenas I e IV estão corretas

Q.11 (0.70) -

```
public class Main {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 3; i++) {  
            for (int j = 1; j <= 2; j++) {  
                System.out.println("Estudante "  
+ i + ", Prova " + j);  
            }  
        }  
    }  
}
```

A execução do código a seguir será bem-sucedida e imprimirá, em tela, 6 linhas.

- a) ☐ Certo
- b) ☐ Errado

Q.12 (0.70) -

```
class Impressora {  
  
    void imprimir(String texto) {  
        System.out.println("Texto");  
    }  
  
    void imprimir(int numero) {  
        System.out.println("Número");  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Impressora i = new Impressora();  
  
        i.imprimir(10);  
    }  
}
```

Assinale a alternativa correta:

- a) ☐ O código imprime 10
- b) ☐ O método imprimir não pode receber inteiro
- c) ☐ O código imprime Número
- d) ☐ Não existe sobrecarga no código
- e) ☐ O código imprime Texto

